

머신러닝

로지스틱 회귀 심화

Chapter 9



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

- 01 다중클래스 분류와 소프트맥스 분류
- 02 사이킷런을 이용한 다중클래스 분류를 코드
- 03 성능 평가

01

다중클래스 분류와 소프트맥스 분류



다중클래스 분류의 개념

- 다중클래스(Multi-Class) : 하나의 데이터가 여러 클래스 중 단 하나의 클래스만 선택되는 분류 문제
- 다중레이블(Multi-Label) : 하나의 데이터가 여러 개의 클래스(레이블)를 동시에 가질 수 있는 분류 문제

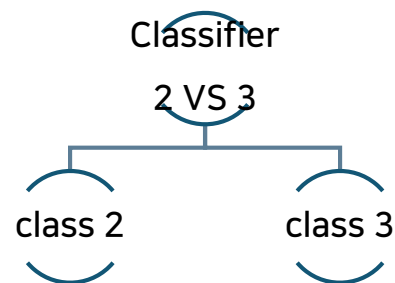
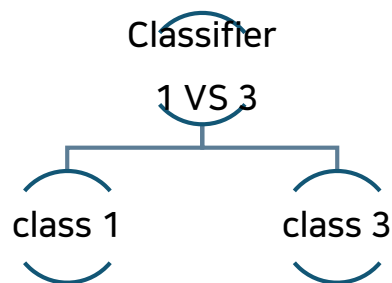
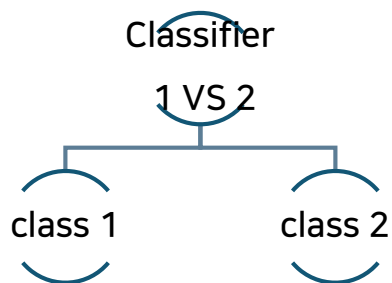
표 10-1 분류 작업에서 다중클래스와 다중레이블의 차이점

분류	다중클래스(multi-class) 분류	다중레이블(multi-label) 분류
작업	2개 이상의 클래스를 가진 분류 작업	상호 배타적이지 않은 속성 예측
중복 선택	중복 선택 불가능 → [1 0 0] 가능, [1 1 0] 불가	중복 선택 가능 → [1 1 0] 가능
예	과일 사진 분류 : 오렌지, 사과, 배	신문기사 분류 : 운동선수-연예인 결혼 기사 → 스포츠/연예 면

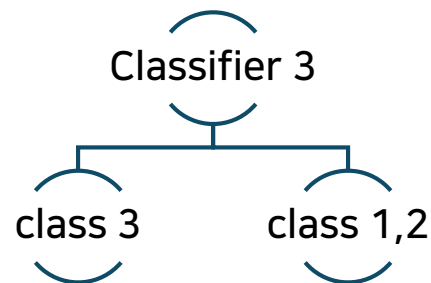
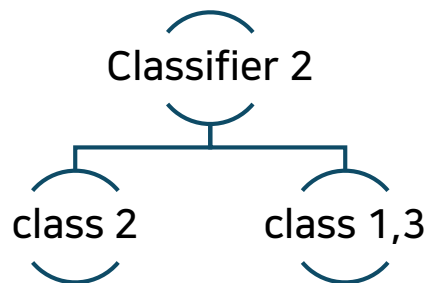
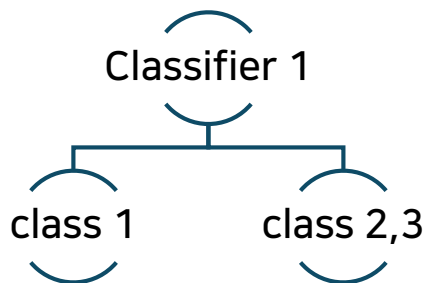


분류 접근

- One-vs-One(OVO): m개의 클래스가 있을 때 클래스 쌍마다 하나의 이진 분류기를 학습하는 방식
 - 총 $\frac{m(m-1)}{2}$ 개만큼의 분류기를 생성하여 최종적으로 각 분류기의 결과를 투표하여 클래스를 결정
 - 분류기가 많아질수록 정확도 높아지지만 비용도 증가

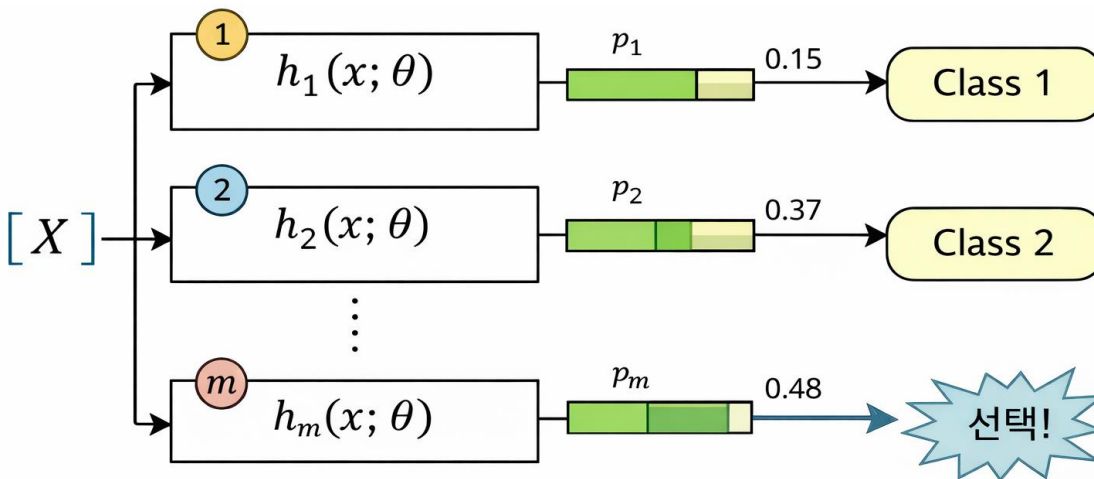
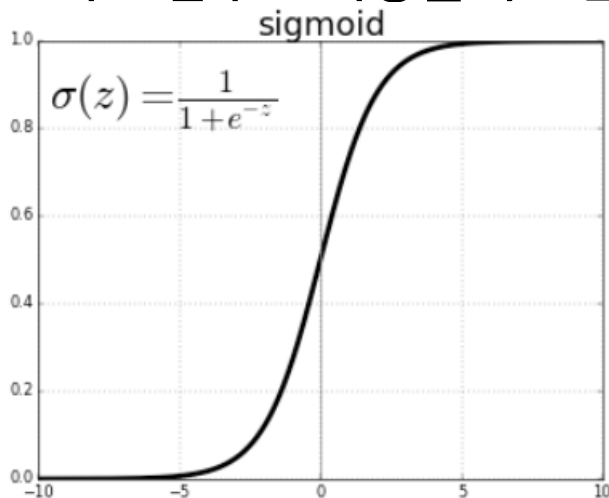


- One-vs-Rest(OVR): 각 클래스를 나머지 전체 클래스와 비교하는 이진 분류기를 여러 개 만드는 방식
 - One-vs-All(OVA)과 같은 의미



▶ 시그모이드 함수 기반 다중 분류의 문제점

- 시그모이드 함수로 다중클래스 분류 문제 다룰 수 있음



- 각각의 클래스에 대한 이진분류기 m 개를 생성한 후 가장 높은 확률이 나오는 클래스를 선택
 - 분류기 번호 m 에 대해 $h_m(x; \theta)$ 로 표현
 - 그러나 $h_m(x; \theta)$ 확률의 합이 1 이상이 된다는 문제 발생
 - 이게 왜 문제? 어떤 분류기에서는 모두 1에 가깝게 예측할 수도 있음
 - 문제 해결 방법은 모든 클래스들의 발생 확률을 1로 정규화



소프트맥스 함수(softmax function)

- 소프트맥스 함수(softmax function) : 다중 클래스 분류에서 각 클래스의 점수 값을 확률 값으로 변환하고 모든 클래스 확률의 합이 1이 되도록 정규화하는 함수

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, 2, 3, \dots, K$$

$$\sum_{j=1}^K \sigma(z)_j = \sum_{j=1}^K P_j = 1$$

- 왜 자연상수로 정규화?
 - 음수 점수도 안전하게 변환됨
 - 지수는 급격한 차이를 강조 -> 모델이 자신 있는 클래스에 높은 확률을 주도록 유도
 - 수학적으로 미분이 간단



소프트맥스 함수 구현

- 각 클래스에 대한 점수 z_i

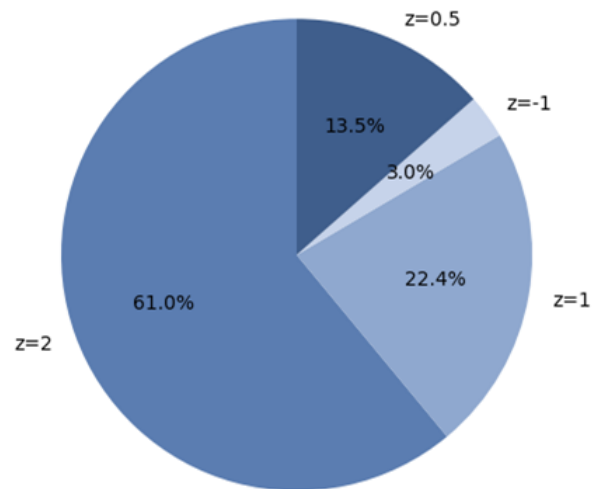
z_j	e^{z_j}	$\frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$
2	7.389	0.609
1	2.718	0.224
-1	0.367	0.030
0.5	1.648	0.135

```
import numpy as np

def softmax(values):
    array_values = np.exp(values)
    return array_values / np.sum(array_values)

values = [2, 1, -1, 0.5]

y = softmax(values)
print(y)
y.sum()
```



```
[0.60946004 0.22420782 0.03034323 0.13598892]
1.0
```


▶ 소프트맥스 함수로 구현하는 다중 클래스 분류

- 오즈에 logit 함수를 붙여 최종적으로 구한 가중치 값

$$\frac{P_j}{1-P_j} \Rightarrow \text{logit}(P_j) = \log_e \left(\frac{P_j}{1-P_j} \right) = z = \theta^T X \quad P$$

$$z = w_0 x_0 + w_1 x_1 + \dots + w_n x_n = \theta^T X$$

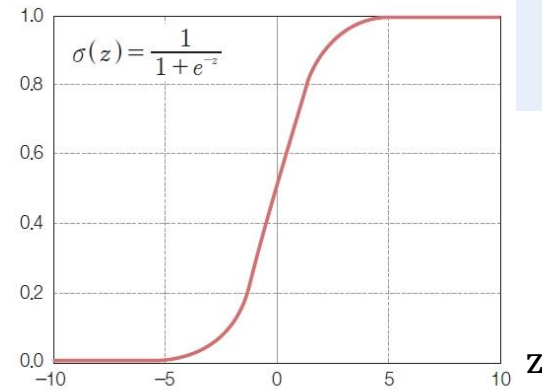


그림 9-4 시그모이드 함수

Ch08 로지스틱 회귀 p.8~9 참고

- 기존의 오즈는 이진분류이지만 다중클래스 분류는 각 클래스 j에 대해 기준 클래스(K)와의 비율을 비교 $\frac{P_j}{P_K}$
 - 다중클래스 분류에서는 j개의 클래스가 있다면 클래스의 개수만큼 가중치에 대한 벡터를 구함
 - 피쳐 벡터와 각 클래스별로 존재하는 가중치 행렬의 선형결합을 z로 나타냄

$$\log_e \left(\frac{P_j}{P_K} \right) = z_j = X^T \theta_j \quad \xrightarrow{\text{양변 } e \text{ 지수}} \quad \frac{P_j}{P_K} = e^{z_j}$$



P_k 와 P_j 정의

P_k

$$\sum_{j=1}^K \frac{P_j}{P_K} = \sum_{j=1}^K e^{z_j} \Rightarrow \frac{1}{P_K} \sum_{j=1}^K P_j = \sum_{j=1}^K e^{z_j} \Rightarrow$$

$$\frac{1}{P_K} \times 1 = \sum_{j=1}^K e^{z_j} \Rightarrow \left(\because \sum_{j=1}^K P_j = 1 \right)$$

$$P_k = \frac{1}{\sum_{j=1}^K e^{z_j}}$$

P_j

$$\frac{P_j}{P_k} = e^{z_j} \Rightarrow \frac{P_j}{\sum_{j=1}^K e^{z_j}} = e^{z_j} \Rightarrow \because P_K = \frac{1}{\sum_{j=1}^K e^{z_j}}$$

$$P_j = \frac{e^{z_j}}{\sum_{j=1}^K e^{z_j}} = \frac{e^{z_j}}{\sum_{j=1}^K e^{x^T \theta_j}} \quad \because z = X^T \theta_j$$

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_3 \end{bmatrix} = \begin{bmatrix} w_{10} & w_{11} & \cdots & w_{1i} \\ w_{20} & w_{21} & \cdots & w_{2i} \\ \vdots & \vdots & \ddots & \vdots \\ w_{j0} & w_{j1} & \cdots & w_{ji} \end{bmatrix}$$



다중 클래스 분류의 가설 함수

- 오른쪽 수식에서 θ 를 학습
 - 각 클래스마다 적절한 θ_j 를 찾기
- 각 가설함수는 각 클래스와 발생확률로 표현 가능

$$h_{\theta}(x) = \begin{bmatrix} P(y=1|x;\theta) \\ P(y=2|x;\theta) \\ \vdots \\ P(y=K|x;\theta) \end{bmatrix} = \frac{1}{\sum_{j=1}^K \exp(\theta^{(j)T}x)} \begin{bmatrix} \exp(\theta^{(1)T}x) \\ \exp(\theta^{(2)T}x) \\ \vdots \\ \exp(\theta^{(K)T}x) \end{bmatrix} \quad P_j = \frac{e^{x^T\theta_j}}{\sum_{j=1}^K e^{x^T\theta_j}}$$

- 최대우도추정법(Maximum Likelihood Estimation, MLE)을 사용해서 P_j 확률을 최대화하는 θ 를 찾기

$$\arg \max_{\theta} \prod_{i=1}^m P(y^{(i)} | x^{(i)}; \theta)$$



다중 클래스 분류의 비용 함수

- 위 수식을 손실(loss)로 생각하여 수식 L로 표현하고 해당 값을 최대화하는 방향으로 정리

$$L = \prod_{i=1}^m P(y^{(i)} | x^{(i)}; \theta) = \prod_{i=1}^m \prod_{j=1}^K p_j^{(i)^{v_{ij}}} \Rightarrow$$

$$-\log L = -\log \prod_{i=1}^m \prod_{j=1}^K p_j^{(i)^{v_{ij}}} = -\sum_{i=1}^m \sum_{j=1}^K v_{ij} \log p_j^{(i)}$$

$$\text{where } v_{ij} = \begin{cases} 1 & \text{if } y^{(i)} \text{ is label } j \\ 0 & \text{if } y^{(i)} \text{ is NOT label } j \end{cases}$$

- 최종적으로 $\frac{\partial l}{\partial z} \frac{\partial z}{\partial \theta}$ 을 구하고자 함

$$l = -\sum_{i=1}^m \sum_{j=1}^K v_{ij} \log p_j^{(i)} \quad \text{where } p_j^{(i)} = \frac{e^{z_j^{(i)}}}{\sum_{j=1}^K e^{z_j^{(i)}}}$$

02

사이킷런을 이용한 다중클래스 분류를 코드



mnist데이터셋의 이해

- 손글씨를 숫자로 인식하는 이미지 분류 문제

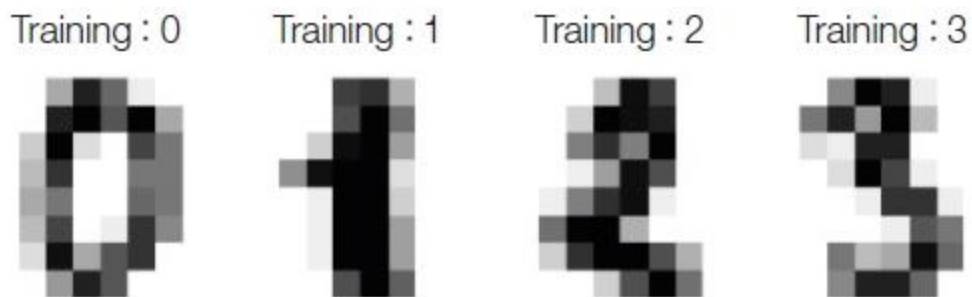
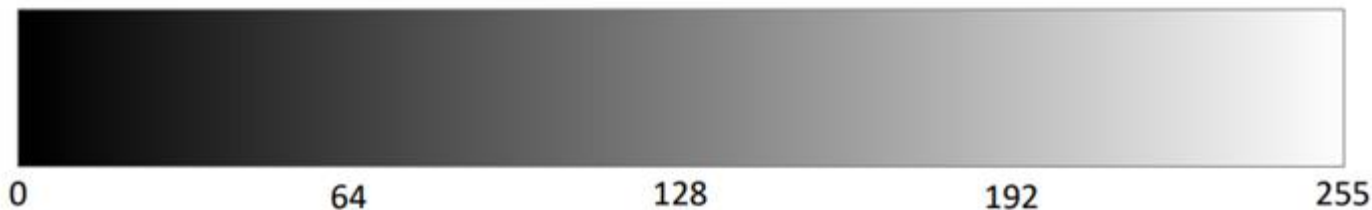


그림 10-1 사이킷런의 mnist 데이터셋 예제

- 컴퓨터는 이미지를 일종의 숫자로 변환하여 인식
 - 이미지를 일종의 점(dot)으로 생각하면 $m \times n$ 만큼의 공간이 존재
 - 그 공간 안에서 색깔이 진할수록 높은 값, 색깔이 옅을수록 낮은 값을 가짐
 - 0~255 값(지금은 반대!)





데이터 불러오기

- datasets 모듈을 호출
- load_digits 함수로 딕셔너리 타입 데이터를 불러오기

```
from sklearn import datasets  
digit_dataset = datasets.load_digits()  
digit_dataset.keys()
```

```
dict_keys(['data', 'target', 'frame', 'feature_names', 'target_names', 'images', 'DESCR'])
```

```
digit_dataset["images"].shape
```

```
(1797, 8, 8)
```

```
digit_dataset["target"][0] #정답
```

```
0
```

```
digit_dataset["images"][0] #데이터
```

```
array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.],  
       [ 0.,  0., 13., 15., 10., 15.,  5.,  0.],  
       [ 0.,  3., 15.,  2.,  0., 11.,  8.,  0.],  
       [ 0.,  4., 12.,  0.,  0.,  8.,  8.,  0.],  
       [ 0.,  5.,  8.,  0.,  0.,  9.,  8.,  0.],  
       [ 0.,  4., 11.,  0.,  1., 12.,  7.,  0.],  
       [ 0.,  2., 14.,  5., 10., 12.,  0.,  0.],  
       [ 0.,  0.,  6., 13., 10.,  0.,  0.,  0.]])
```



데이터 확인

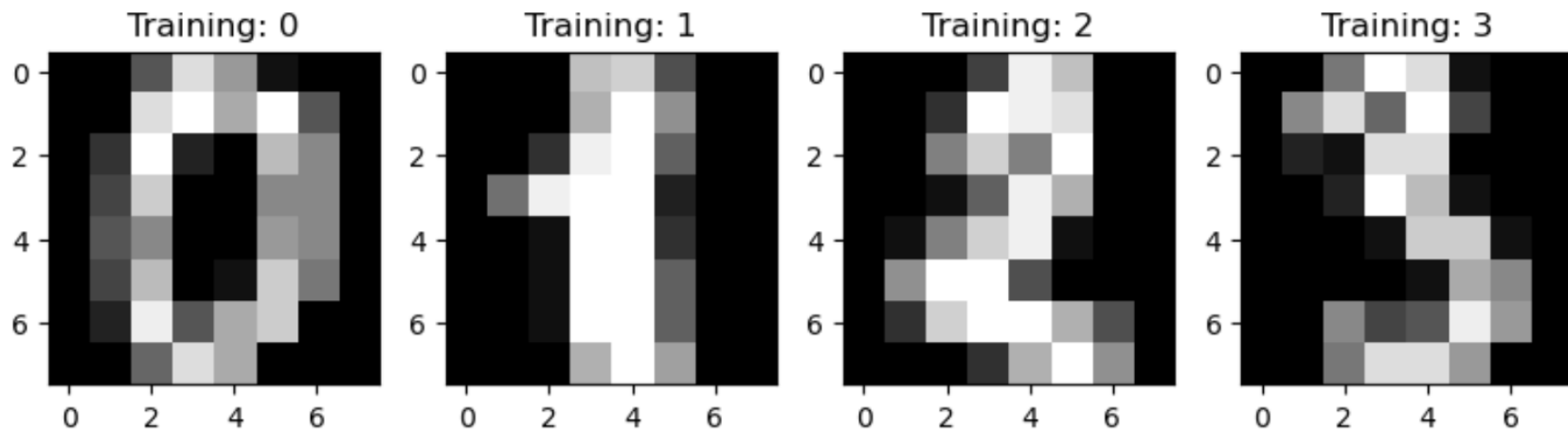
```
import matplotlib.pyplot as plt

_, axes = plt.subplots(nrows=1, ncols=4,
figsize=(10, 3))

for i, ax in enumerate(axes):

    image = digit_dataset["images"][i]
    label = digit_dataset["target"][i]

    ax.set_title('Training: %i' % label)
    ax.imshow(image, cmap='gray')
```





데이터 분류

- 데이터가 8×8 행렬이므로 2D 이미지로 표현되었지만 다음 코드와 같이 총 64개의 피쳐(벡터 형태)를 가진 하나의 데이터로 받을 수 있음

```
digit_dataset["data"][0].shape
```

(64,)

- 데이터를 훈련 데이터셋과 테스트 데이터셋으로 구분

```
from sklearn.model_selection import train_test_split
```

```
X = digit_dataset["data"]
```

```
y = digit_dataset["target"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y)#default: test_size=0.25
```

```
X_train.shape , X_test.shape
```

((1347, 64), (450, 64))



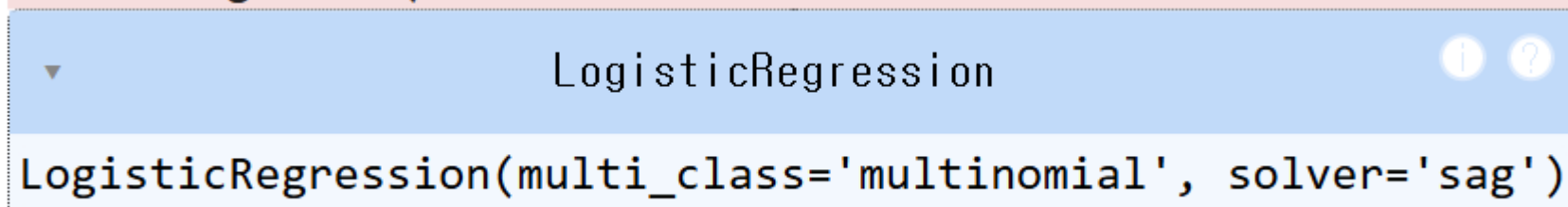
모델 생성

- Multi_class 매개 변수
 - ovr : 클래스 모드를 모두 이진모델로 만들어 학습
 - multinomial : 소프트맥스 함수를 사용하여 계산하는 방식
- solver 매개변수: Stochastic Average Gradient(SAG)으로 변경

```
from sklearn.linear_model import LogisticRegression

logreg_ovr = LogisticRegression(multi_class="ovr")
logreg_softmax = LogisticRegression(multi_class="multinomial", solver="sag")

logreg_ovr.fit(X_train, y_train)
logreg_softmax.fit(X_train, y_train)
```



LogisticRegression

```
LogisticRegression(multi_class='multinomial', solver='sag')
```



성능 평가

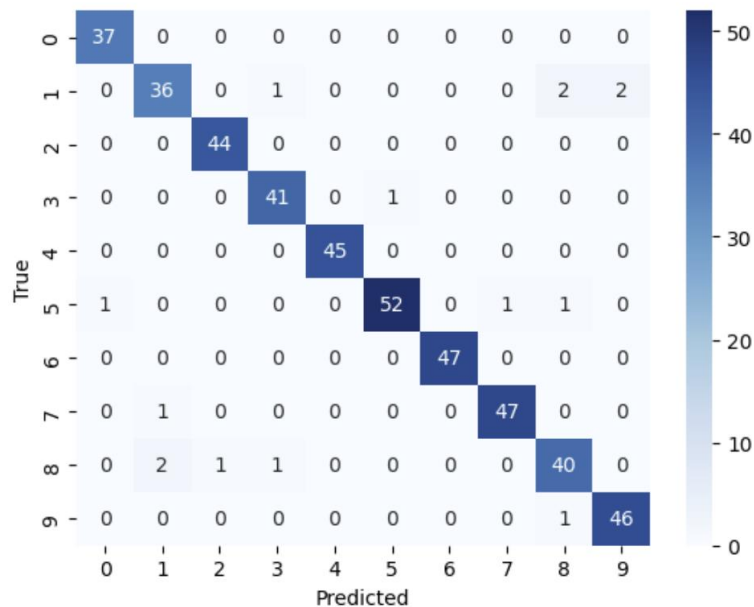
- 일반적으로 다중클래스 분류도 기존 혼동행렬을 사용
- 각 클래스 대비 예측한 값을 행렬 형태로 표현

```
from sklearn.metrics import confusion_matrix
y_pred = logreg_ovr.predict(X_test).copy()
y_true = y_test.copy()
cm = confusion_matrix(y_true, y_pred)
cm
```

```
array([[37,  0,  0,  0,  0,  0,  0,  0,  0,  0],
       [ 0, 36,  0,  1,  0,  0,  0,  0,  2,  2],
       [ 0,  0, 44,  0,  0,  0,  0,  0,  0,  0],
       [ 0,  0,  0, 41,  0,  1,  0,  0,  0,  0],
       [ 0,  0,  0,  0, 45,  0,  0,  0,  0,  0],
       [ 1,  0,  0,  0,  0, 52,  0,  1,  1,  0],
       [ 0,  0,  0,  0,  0,  0, 47,  0,  0,  0],
       [ 0,  1,  0,  0,  0,  0,  0, 47,  0,  0],
       [ 0,  2,  1,  1,  0,  0,  0,  0, 40,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  1, 46]], dtype=int64)
```

```
import seaborn as sns
```

```
sns.heatmap(cm, annot=True, cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```





성능 평가

- 라벨별로 분류 성능을 수치화하여 표시

- support : 실제 샘플 개수 (ex. '0' 샘플 수 47개)
- macro avg: 평균의 평균(모든 클래스가 동일한 중요도)
- weighted avg: 가중 평균(ex. F1스코어의 가중평균 ▶ $F1_{class1} * W_1 + F1_{class2} * W_2 + \dots + F1_{classN} * W_N$)

```
from sklearn.metrics import classification_report  
print(classification_report(y_true, y_pred))
```

	precision	recall	f1-score	support
0	0.97	1.00	0.99	37
1	0.92	0.88	0.90	41
2	0.98	1.00	0.99	44
3	0.95	0.98	0.96	42
4	1.00	1.00	1.00	45
5	0.98	0.95	0.96	55
6	1.00	1.00	1.00	47
7	0.98	0.98	0.98	48
8	0.91	0.91	0.91	44
9	0.96	0.98	0.97	47
accuracy			0.97	450
macro avg	0.97	0.97	0.97	450
weighted avg	0.97	0.97	0.97	450

03 성능 평가



정밀도와 민감도의 트레이드오프

- 정밀도(precision)와 민감도(recall)는 일반적으로 둘 다 동시에 상승하기 어렵고 임계값(threshold)에 따라 변화가 일어남
 - 두 값을 모두 고려하여 성능을 측정하기 쉽지 않음
 - 임계값을 낮추면 (예: 0.3 이상이면 P로 본다)
더 많은 샘플을 P로 예측하게 됨 → TP도 많아짐, FP도 많아짐
→ Precision ↓ Recall ↑
 - 임계값을 올리면 (예: 0.9 이상이어야 P로 본다)
정말 자신 있는 것만 P로 예측 → FP 줄어듦
→ Precision ↑ Recall ↓
- ROC 커브 : 분류기의 임계값을 변화시키면서 FPR(False Positive Rate)과 TPR(True Positive Rate)의 관계를 나타낸 그래프
 - 'Receiver Operating Characteristics'의 약자

$$PRECISION(PPV) = \frac{TP}{TP + FP}$$

$$RECALL(TPR) = \frac{TP}{TP + FN}$$

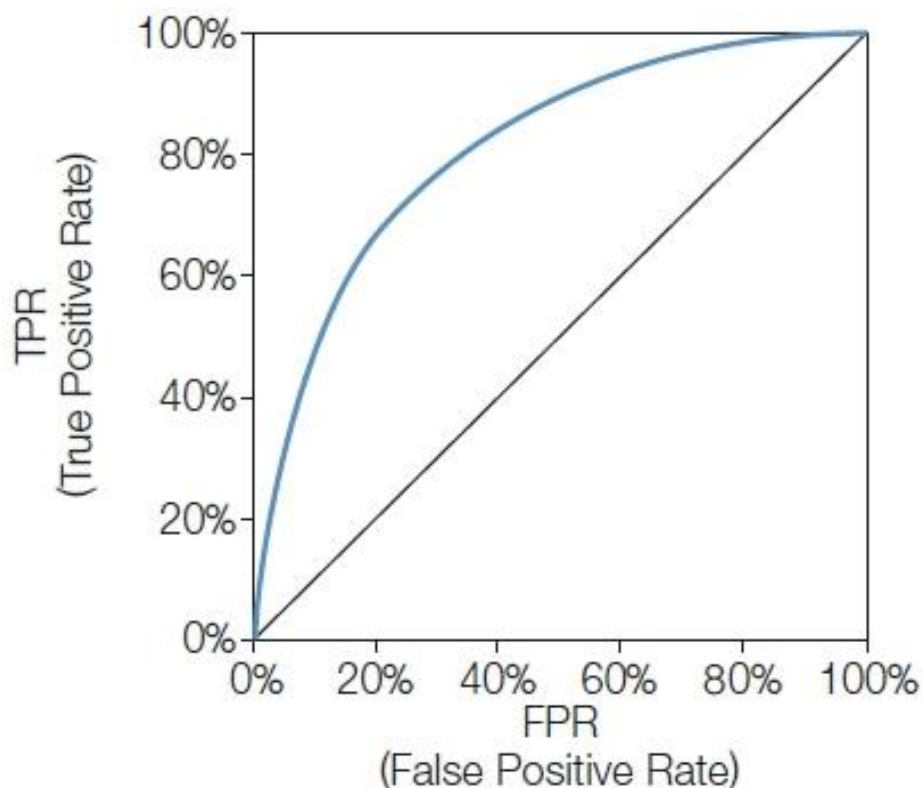


ROC 커브 표현

- TPR(True Positive Rate)과 FPR(False Positive Rate)을 각각 y축, x축에 나타내어 그래프를 작성

$$Recall = TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$



데이터	클래스	임계값
1	P	0.9
2	P	0.8
3	N	0.7
4	P	0.6
5	P	0.55
6	N	0.54
7	N	0.53
8	N	0.51
9	P	0.5
10	N	0.4



ROC 커브 표현

표 10-3 데이터 정리

데이터	클래스	임계값	TP	FP	TN	FN	TPR	FPR
1	P	0.9	1	0	5	4	0.2	0
2	P	0.8	2	0	5	3	0.4	0
3	N	0.7	2	1	4	3	0.4	0.2
4	P	0.6	3	1	4	2	0.6	0.2
5	P	0.55	4	1	4	1	0.8	0.2
6	N	0.54	4	2	3	1	0.8	0.4
7	N	0.53	4	3	2	1	0.8	0.6
8	N	0.51	4	4	1	1	0.8	0.8
9	P	0.5	5	4	0	1	1	1
10	N	0.4	5	5	0	0	1	1

$$Recall = TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

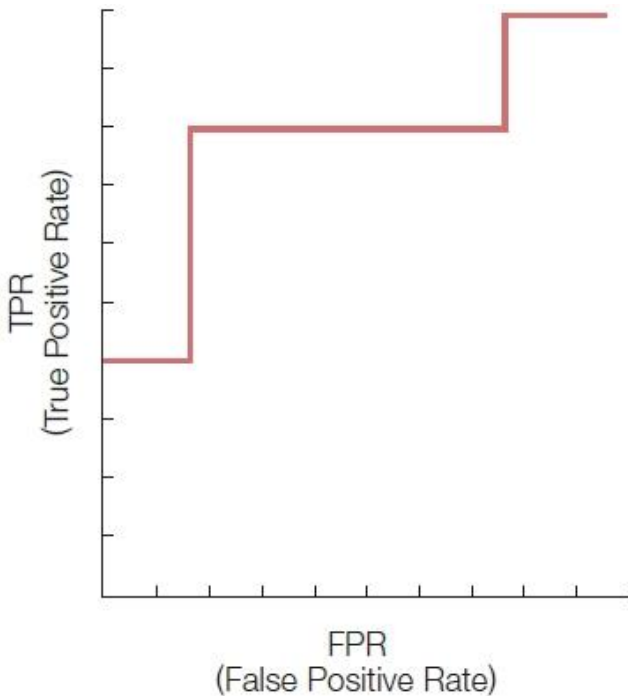


그림 10-3 [표 10-3]을 기준으로 한 ROC 커브



AUC(Area Under Curve)

- AUC(Area Under Curve) : ROC 곡선 아래 면적을 의미하며 분류 모델의 전체적인 성능을 나타내는 지표

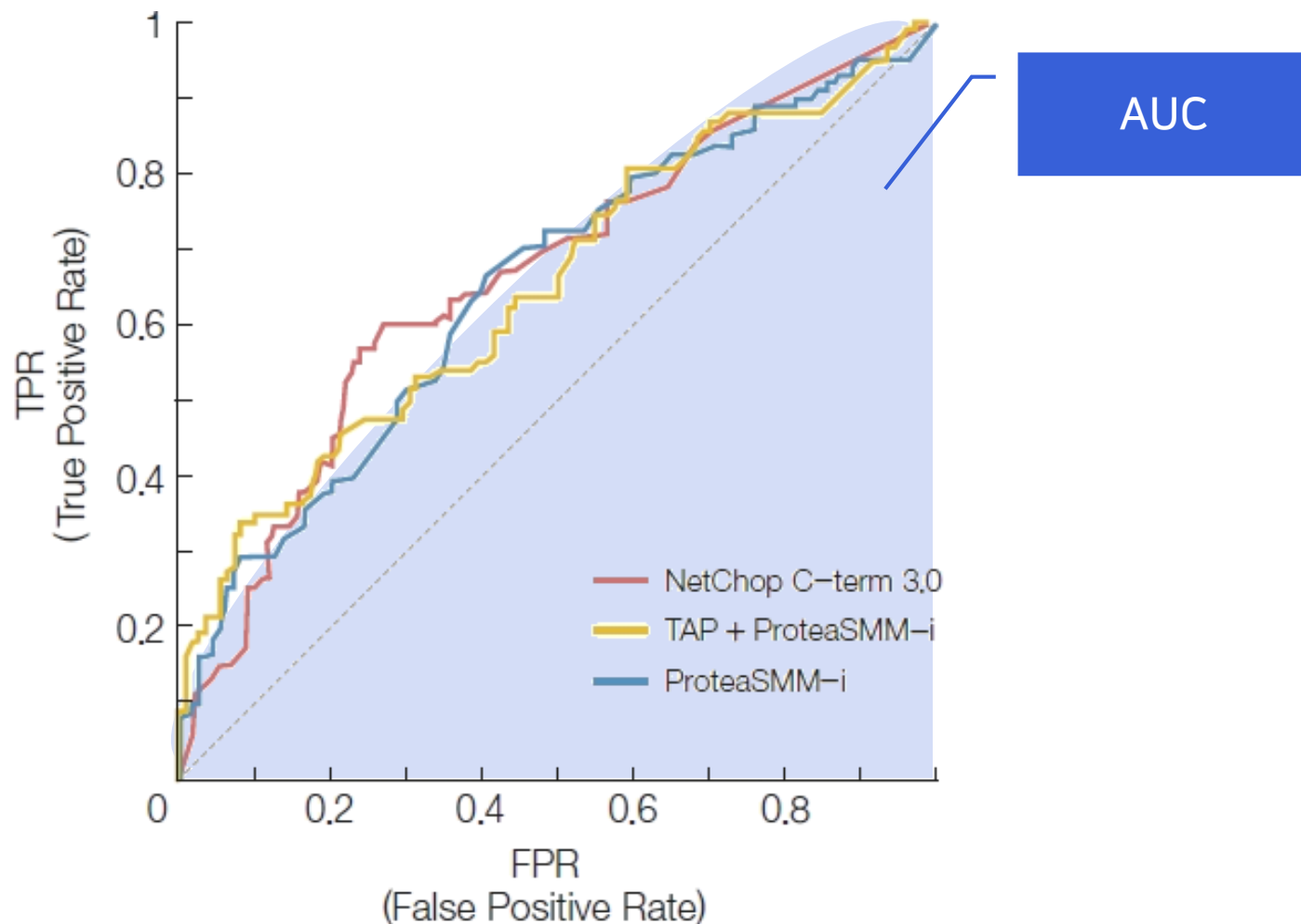


그림 10-4 여러 모델의 성능을 상대적으로 비교하기 위한 그래프



ROC 커브와 AUC 코드

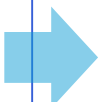
- 정답 y 값과 각 데이터 별 positive 클래스(양성 클래스)의 예측 확률
- ROC 커브 함수인 roc_curve로 fpr, tpr, thresholds 반환
 - pos_label 매개변수: ROC에서 Positive로 사용할 클래스 값
- auc 함수는 FPR과 TPR로 구성된 ROC 곡선 아래의 면적을 계산하여 모델의 분류 성능을 평가

```
import numpy as np
from sklearn.metrics import roc_curve, auc

y_true = np.array([0, 0, 1, 1])# 실제 클래스 (0 = negative, 1 = positive)
pred = np.array([0.1, 0.4, 0.35, 0.8])# positive 클래스(양성 클래스)의 예측 확률

fpr, tpr, thresholds = roc_curve(y_true, pred, pos_label=1)
roc_auc = auc(fpr, tpr)

# 결과 출력
print("FPR:", fpr)
print("TPR:", tpr)
print("Thresholds:", thresholds)
print("AUC:", roc_auc)
```



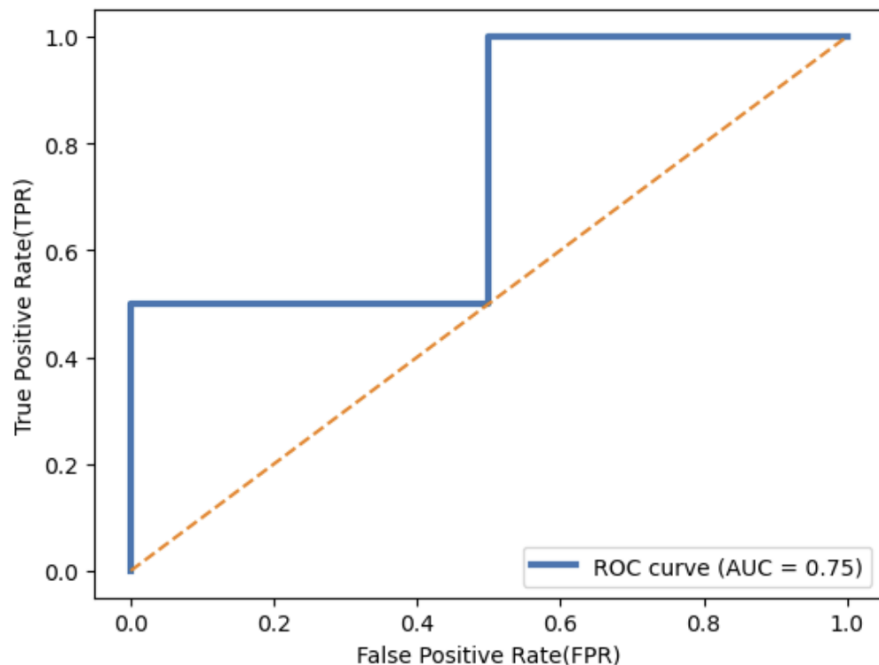
```
FPR: [0.  0.  0.5 0.5 1. ]
TPR: [0.  0.5 0.5 1.  1. ]
Thresholds: [ inf 0.8  0.4  0.35 0.1 ]
AUC: 0.75
```



ROC 커브와 AUC 그래프

```
import matplotlib.pyplot as plt

plt.plot(fpr, tpr, lw=3, label='ROC curve (AUC = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel('False Positive Rate(FPR)')
plt.ylabel('True Positive Rate(TPR)')
plt.legend(loc="lower right")
plt.show()
```



다중 클래스 분류에서의 ROC 커브와 AUC 코드

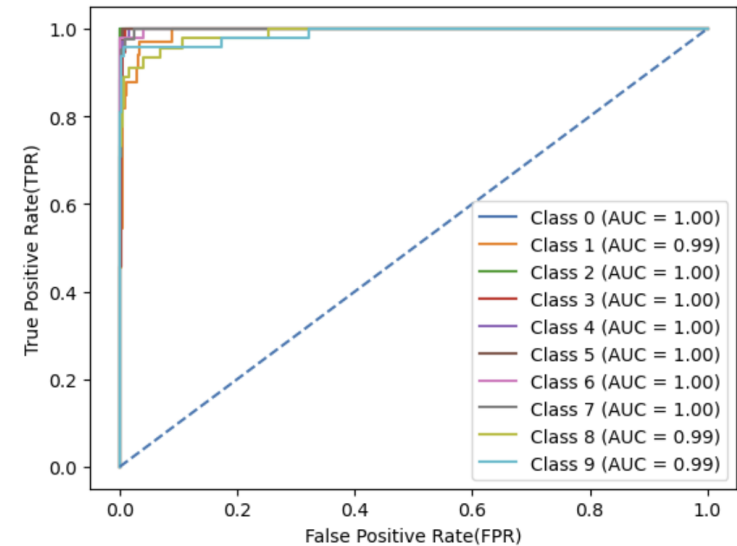
```
import pandas as pd

y_bin = pd.get_dummies(y_test).values#실제값을 one-hot 형태로 변환
pred = logreg_ovr.predict_proba(X_test)

#클래스 수
n_classes = pred.shape[1]

#클래스별 ROC 계산
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_bin[:, i], pred[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
```





SUMMARY

- 다중클래스 분류: 여러 클래스 중 단 하나의 클래스만 선택되는 분류 문제
 - 이진분류로 구현: OVR, OVO
 - 소프트맥스 함수로 분류: 클래스 점수를 확률로 변환하고 전체 합이 1이 되도록 정규화
- 사이킷런의 LogisticRegression 함수로 OVR 및 소프트맥스 기반 다중클래스 분류를 구현할 수 있음
- 성능평가
 - 혼동행렬과 classification report로 분류 성능을 평가할 수 있음
 - ROC 커브: FPR과 TPR의 관계
 - 다중클래스 ROC는 각 클래스를 One-vs-Rest 방식으로 계산할 수 있음
 - AUC: ROC 곡선 아래 면적을 의미함



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝